

JigsawRL: Assembling RL Pipelines for Efficient LLM Post-Training

Zhengding Hu, Hehua Ouyang, Chang Chen, Zaifeng Pan, Yue Guan,
Zhongkai Yu, Zhen Wang, Steven Swanson, Yufei Ding
University of California, San Diego

Abstract

We present JigsawRL, a cost-efficient framework that explores *Pipeline Multiplexing* as a new dimension of RL parallelism. JigsawRL decomposes each pipeline into a *Sub-Stage Graph* that exposes the intra-stage and inter-worker imbalance hidden by stage-level systems. On this abstraction, JigsawRL resolves multiplexing interference through dynamic resource allocation, eliminates fragmented utilization by migrating long-tail rollouts across workers, and formulates their coordination as a graph scheduling problem solved with a look-ahead heuristic. On 4-64 H100/A100 GPUs across different agentic RL pipelines and models, JigsawRL achieves up to $1.85\times$ throughput over Verl on synchronous RL, $1.54\times$ over StreamRL and AReaL on asynchronous RL, and supports heterogeneous pipelines with moderate latency trade-off.

1 Introduction

Reinforcement learning (RL) [34] has become a standard technique for LLM post-training. Unlike the one-time process of creating a foundation model, RL is often applied iteratively to adapt a single pretrained model into various specialized versions. By continuously collecting feedback on specific task sets, RL allows researchers to branch a base model into different directions, such as enhancing complex reasoning [16, 21, 43], agentic behaviors [8, 25, 53], or tool-usage capabilities [12]. For example, within the Qwen model family (0.5–72B parameters) alone, the community has already published approximately 180,000 RL fine-tuned variants [2].

With such wide application, the **cost efficiency** of RL pipelines has emerged as a critical concern for both research and production. Yet, existing RL frameworks [1, 45, 71] primarily optimize for time efficiency, while overlooking cost efficiency as a first-class objective. Our profiling on Verl [45] reveals that improving time efficiency does not necessarily translate to better cost efficiency. As shown in Figure 1(a), scaling up GPU resources increases pipeline throughput, but leads to rapidly growing monetary costs with diminishing returns. This inefficiency stems from **resource underutilization**

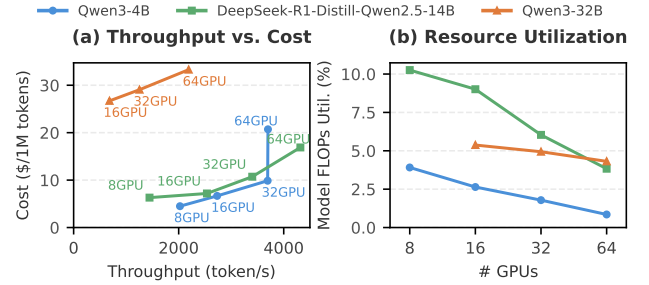


Figure 1: (a) Trade-off between pipeline throughput and monetary cost² across different models, spanning from base models [57] to partially tuned variants [16]. (b) Pipeline resource utilization, measured by FLOPs utilization (MFU).

in RL pipelines. Figure 1(b) demonstrates that the average GPU utilization, measured as MFU, is below 10%. As the number of GPUs increases, MFU degrades even further.

This cost inefficiency mainly comes from **workload imbalance in the rollout stage** [14, 69]. Due to the strict synchronization between alternating rollout and training, each step is gated by the slowest samples. As shown in Figure 2(a), a small number of long sequences dominate the rollout step, causing the effective batch size to shrink during decoding and thus reducing GPU utilization. This imbalance is further exacerbated by the emerging trend of agentic workloads, where multi-turn reasoning [8, 25] and external tool-calling [12, 23] introduce highly stochastic and long-tailed sequence distributions.

Existing RL systems explore asynchronous execution [13, 44, 68] and time multiplexing [55, 56, 69] to improve the efficiency, but they still fail to eliminate the workload imbalance both within and between stages. As shown in Figure 2(b), one-step off-policy execution such as StreamRL [68] overlaps rollout and training across workers, but the imbalance within

²Monetary costs are estimated based on the on-demand pricing of AWS EC2 A100 instances [4] (\$4.10 per GPU hour).

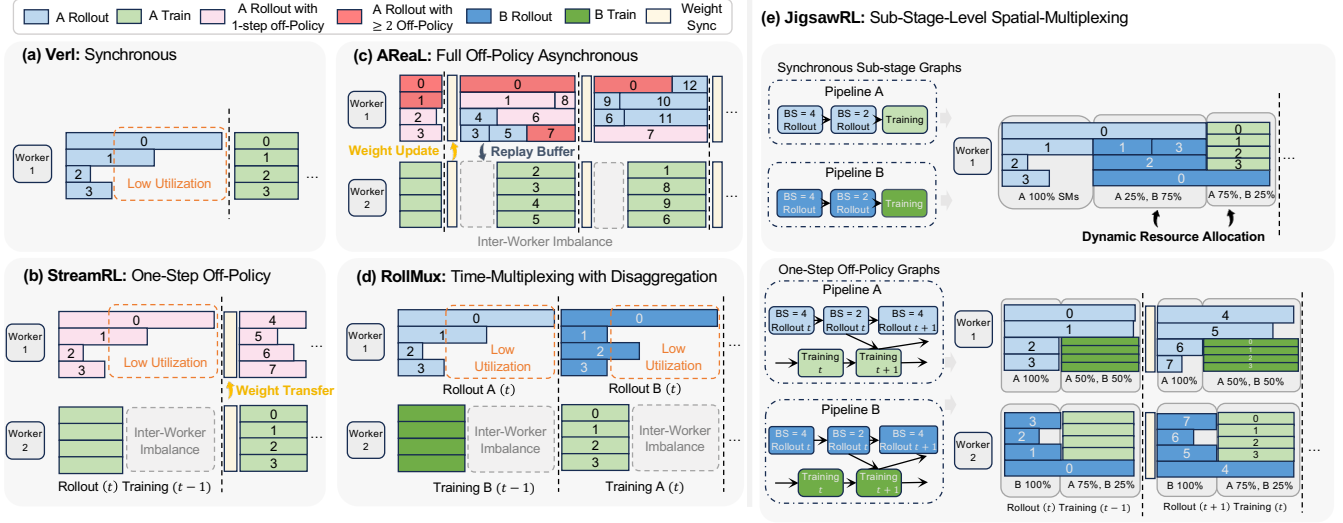


Figure 2: Comparison of execution behaviors under existing RL frameworks (a-d) and JigsawRL’s sub-stage-level spatial multiplexing (e). Existing approaches suffer from low utilization and inter-worker imbalance. JigsawRL multiplexes sub-stages with complementary resource demands from different pipelines to improve GPU utilization and thus cost efficiency.

rollout still remains. Figure 2(c) shows fully asynchronous execution like AReal [13], which further relaxes synchronization but introduces imbalance across workers as rollout and training progress at different speeds. With these training and rollout disaggregation settings [14, 44, 68], partial resource reallocation helps mitigate imbalances across workers, but introduces additional overheads for frequent weight transfer. Moreover, off-policy methods introduce data staleness, which can affect convergence and stability [66]. Figure 2(d) shows time multiplexing such as RollMux [55], which enables concurrent pipelines with disaggregated workers, but imbalance within each rollout stage and across workers still exists.

The above works are still constrained by the dependencies and imbalance within a single RL pipeline, and GPU utilization still has room for improvement. In this work, we explore a new dimension of RL parallelism: *Spatial Multiplexing of concurrent RL workloads*. We leverage multiple concurrent workloads in shared clusters and serverless environments [33, 54, 61]. This multi-workload setting provides more diverse stages and more concurrent execution opportunities, which better fills low-utilization periods.

However, achieving efficient multiplexing is challenging due to the complex and dynamic execution behaviors in RL pipelines, making compute and memory resource demand vary over time. Naive approaches such as static resource partitioning [30, 31] or reusing existing inference multiplexing frameworks [11, 64] either lead to resource contention or fail to fill low-utilization periods effectively. Efficient multiplexing thus requires modeling these dynamic behaviors to enable resource coordination and pipeline scheduling.

To this end, we propose JigsawRL, a cost-efficient and adaptive RL multiplexing system. The core abstraction is the *Sub-Stage Execution Graph*, which decomposes coarse-grained stages (e.g., rollout and training) into fine-grained sub-stages with distinct compute and memory characteristics, exposing intra-stage and inter-worker imbalance that stage-level systems cannot see. Based on this abstraction, JigsawRL formulates pipeline execution as two graph operations. *Sub-stage Multiplexing* enables concurrent execution of sub-stages across pipelines with dynamic resource allocation. *Sub-stage Merging* migrates long-tail rollout samples across DP instances to eliminate fragmented execution. A look-ahead heuristic coordinates both operations along the critical path.

In summary, our main contributions are as follows:

- We propose the *Sub-Stage Graph* abstraction that exposes intra-stage and inter-worker imbalance hidden by stage-level systems, turning pipeline multiplexing into a graph construction and scheduling problem.
- We observe that RL sub-stages exhibit complementary compute and memory demands, and exploit this through *sub-stage multiplexing* with dynamic resource allocation.
- We observe that long-tail rollouts fragment GPU utilization across DP workers, and address this through *sub-stage merging* with sample migration and balancing.
- On up to 64 GPUs across three agentic RL pipelines and six models, JigsawRL achieves up to $1.85\times$ throughput over synchronous and $1.54\times$ over asynchronous baselines, and supports heterogeneous pipelines.

2 Background and Related Work

2.1 Reinforcement Learning for LLMs

Reinforcement learning (RL) has become an essential technique for aligning human preferences [5, 34] and enhancing capabilities [16, 21]. Through iterative multi-stage training loops, RL enables models to obtain feedback from reward mechanisms and continuously refine their generation policies. Therefore, optimizing the efficiency of RL pipelines is critical for meeting practical deployment demands.

Modern RL systems adopt a multi-stage pipeline design. A typical pipeline processes each global batch sequentially through several stages. The *Rollout* stage generates responses from the current model. The *Reference* stage computes log probabilities using a frozen model to regularize policy updates. The *Reward* stage evaluates outputs using a reward model [34] or rule-based signals [16]. The *Training* stage performs backpropagation and updates model parameters. Recent agentic pipelines [12, 23] further include a *Tool* stage to incorporate external feedback.

The performance bottleneck of RL pipelines varies across workloads. Table 1 shows the stage time breakdown of Verl [45] under different models and tasks. We observe three key patterns. First, model capability and task difficulty affect stage composition: weaker models or harder tasks lead to longer reasoning chains and increase the proportion of rollout. Second, scaling GPUs shifts bottlenecks across stages. Due to limited scalability and sensitivity to imbalance, rollout becomes dominant at larger scales. Third, agentic workloads introduce additional overhead from tool usage, which can account for a large portion of execution time. Overall, these results show that bottlenecks are highly workload-dependent, requiring coordinated orchestration across stages.

2.2 RL Frameworks

Building efficient RL systems for LLMs has attracted considerable research attention. Existing frameworks [1, 14, 15, 17, 19, 28, 36, 45, 51, 55, 56, 60, 69, 71] adopt a tightly coupled design that leverages high-performance LLM serving engines [24, 67] for the rollout stage and training engines [46, 65] for the training stages, while incorporating various optimizations for complex execution pipelines and stage orchestration. **Parallelism Optimization for Synchronous RL.** Standard RL pipelines operate in a strictly synchronous and sequential manner, alternating between rollout and training stages. Frameworks such as Verl [45] and NeMo-RL [1] allow independent parallelism configurations for the two engines by dynamically switching execution contexts at runtime and performing parameter resharding. ReaL [28] further formulates multi-stage pipeline deployment as a graph search problem, automatically identifying optimal parallelism configurations. Disaggregated frameworks [19, 47, 51] decouple rollout and

Table 1: Stage time proportion across different RL pipeline configurations with GRPO [43]. In our evaluated workload, the Reward stage is rule-based [16] with negligible overhead.

Model Size	Task	#GPUs	Rollout	Reference	Training	Tool
Qwen3-0.6B	GSM8K	4	66.1%	14.2%	19.7%	—
Qwen3-4B	GSM8K	4	52.2%	18.7%	29.1%	—
Qwen3-4B	AIME	4	80.9%	6.9%	12.2%	—
Qwen3-4B	GSM8K	64	67.4%	15.8%	16.8%	—
Qwen3-4B	Search-R1	4	39.1%	16.6%	24.6%	19.7%

training engines across separate GPU clusters to avoid step-wise context switching, but incur additional overheads such as GPU idling and communication for weight synchronization. Specific pipeline optimizations have also been proposed, including rollout length-based scheduling [14, 36] and speculative decoding [7, 17, 26, 42, 50], to mitigate rollout imbalance. **Asynchronous RL.** Synchronous RL suffers from imbalanced rollout (see Section 3.1) due to long-tail samples, which leads to underutilized GPU resources. Off-policy algorithms [29, 39] break the strict dependency between stages, enabling asynchronous rollout and training so that idle GPUs can proceed with subsequent-stage computation instead of blocking. *AReal* [13] improves pipeline efficiency by discarding overlong samples and recomputing them later to mitigate the long-tail effect. *Laminar* [44] proposes fully asynchronous rollout and trainer instances to break barriers between stages, leveraging relay buffers to support fine-grained weight updates and isolate long-tail samples. *RLinf* [62] enables more flexible data and stage partitioning at a finer granularity, achieving dynamic spatiotemporal scheduling within a single RL pipeline. However, asynchronous RL suffers from data staleness [66], which can degrade training stability and convergence. Such trade-offs are undesirable in many real-world deployments, where strict correctness and stability requirements must be met, making it still critical to address inefficiencies within synchronous RL pipelines.

Overall, existing works focus on optimizing an *individual RL training pipeline* for a single LLM. In contrast, this paper explores a new dimension by *multiplexing RL pipelines* to improve resource utilization. This introduces additional flexibility by enabling stages with diverse resource demands to share GPU resources, while remaining orthogonal to existing single-pipeline optimization methods.

3 Motivation

3.1 RL Workload Imbalance

RL training exhibits highly imbalanced and dynamic GPU utilization patterns, primarily driven by the rollout stage. We profile the rollout GPU utilization overtime with GRPO [43] across different datasets and models, as shown in Figure 3. This imbalance comes from **diverse agentic behaviors**, including **variation in single-turn decoding lengths**, **interleaved**

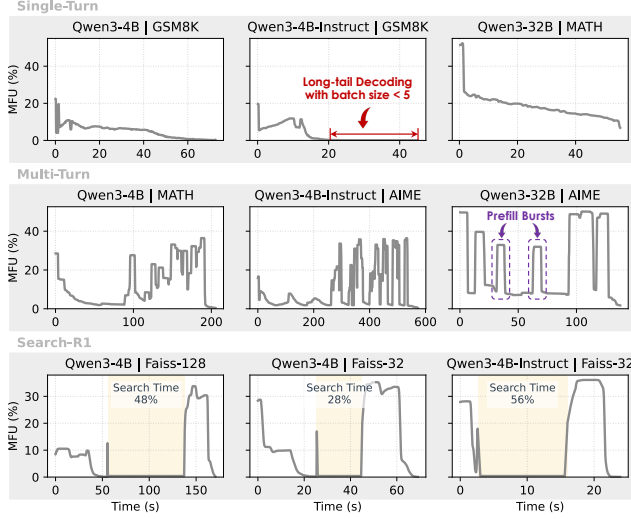


Figure 3: Different agentic behaviors in rollout stages and the resulting highly dynamic and imbalanced GPU utilization.

prefill and decoding in multi-turn interactions, and GPU idle with external tool usage.

Varied Decoding Lengths. Decoding lengths are highly imbalanced across samples within a batch [14], leading to a long-tail effect where a small number of long sequences determine the overall latency. As shorter sequences finish early, the effective batch size quickly shrinks, leaving the GPU underutilized for a large portion of the execution.

We also find that the imbalance is closely associated with model capability and task difficulty. Easier tasks or stronger models tend to produce shorter responses on average, but also more skewed length distributions. For example, under the same model size (Qwen3-4B), the more capable instruct-tuned variant (Qwen3-4B-Instruct) produces shorter responses on GSM8K, but with a more skewed length distribution, causing the GPU to operate at very small effective batch sizes for over 50% of the decoding time.

Multi-turn Rollout Idling. In agentic RL workloads, sampling exhibits multi-turn iterative behaviors [3, 25, 41, 49, 53]. This leads to an interleaving pattern of prefill and decoding phases: Prefill is no longer a one-shot initialization stage. Instead, after a period of decoding, new turns trigger additional prefill requests. Thus, GPU utilization exhibits a periodic rise-and-fall, with sharp utilization spikes during compute-intensive prefill bursts, followed by lower, memory-bound utilization during decoding.

We also observe imbalanced distribution in the number of turns across different samples: while most samples complete within 1–2 turns, a small fraction may extend to 4–5 turns. Such skewness further amplifies utilization imbalance.

GPU Idle with Tool Usage. In agentic RL pipelines [12, 23, 40], coordinating LLM execution with external tools intro-

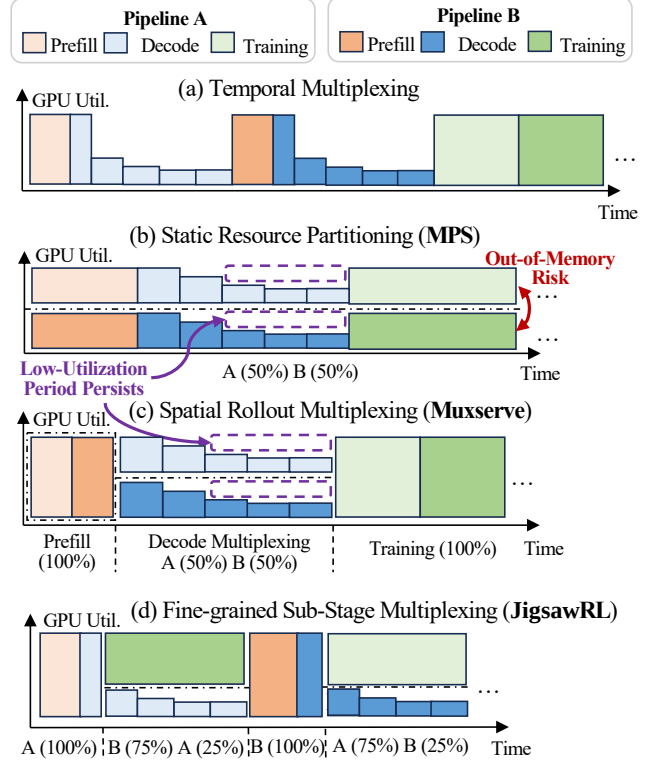


Figure 4: Comparison across different multiplexing methods.

duces additional utilization imbalance. Tool usage typically runs on CPUs or relies on remote services, during which GPUs remain idle while waiting for responses. As tool latency increases (e.g., in RAG workloads), such idle periods can account for over 50% of the total execution time.

Tool usage also introduces new context, triggering additional prefill bursts. We observe that tool outputs are themselves highly imbalanced. For example, retrieved document lengths in RAG can vary by over 50%, which further amplifies sequence length imbalance across requests.

Moreover, blocked tool calls introduce long-tail effects due to the extra synchronization, as execution must wait for all requests to generate their tool calling instructions. Blocked calling is common for heavy tools (e.g., database queries [10, 20], sandboxed execution [22, 59, 70], or search APIs [35, 37]), where single-batched tool execution is more efficient.

3.2 Challenges in RL Multiplexing

Given the highly imbalanced and dynamic GPU utilization observed in RL pipelines, we explore an additional optimization dimension: *Improve utilization by multiplexing different RL pipelines.* By multiplexing pipelines, stages with different utilization characteristics (e.g., compute-intensive prefill/training and memory-bound long-tail decoding) can share

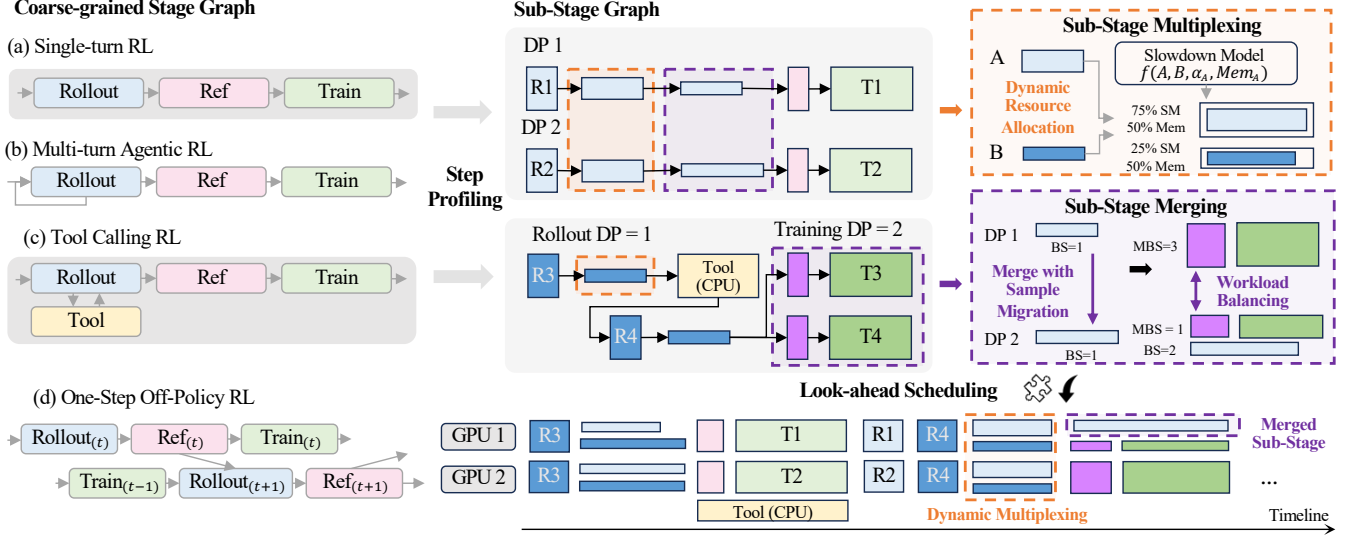


Figure 5: Overview of JigsawRL. Starting from coarse-grained pipeline graphs, JigsawRL constructs fine-grained sub-stage graphs, and enables efficient concurrent pipeline execution through sub-stage multiplexing, merging and graph scheduling.

the GPU resources, thereby improving overall resource utilization and cost-efficiency. Such opportunities commonly arise in practical settings where RL workloads from multiple users are executed concurrently on shared GPU clusters, cloud servers, or serverless environments [33, 48].

However, achieving efficient multiplexing for RL pipelines remains challenging. As illustrated in Figure 4, existing approaches are insufficient to handle the complex and dynamic stage orchestration. Temporal multiplexing (a) executes different stages sequentially, and thus fails to utilize idle resources within each stage. Spatial multiplexing (b), e.g., via MPS [30] or MIG [31], enables concurrent execution of multiple pipelines, but naively multiplexing full pipelines can lead to severe memory pressure. Memory-consuming training stages may overlap and cause out-of-memory (OOM) failures. Directly adopting LLM inference multiplexing systems [11] enables spatial rollout multiplexing (c). However, concurrent execution of the long-tail, memory-bound rollout stage still leads to low GPU utilization. Meanwhile, the training stage remains serialized because of the huge memory consumption, resulting in significantly increased per-pipeline latency.

In contrast, JigsawRL (d) enables more fine-grained multiplexing by explicitly identifying and co-scheduling resource-complementary stages, while dynamically adapting resource allocation. This design effectively mitigates both resource contention and long-tail inefficiencies, achieving a better balance between overall throughput and pipeline latency.

4 System Design

We propose JigsawRL, a cost-efficient and adaptive RL multiplexing framework. JigsawRL introduces a graph-based abstraction. This abstraction models the RL pipeline into fine-grained sub-stages to capture utilization imbalance. Based on this abstraction, JigsawRL formulates pipeline multiplexing as two graph operations to improve resource utilization and mitigate contention. *Sub-stage Mux* enables concurrent execution of complementary sub-stages across pipelines through dynamic resource allocation. *Sub-stage Merge* aggregates low-utilization long-tail samples across DP instances to eliminate fragmented execution. JigsawRL finally applies a heuristic look-ahead scheduling algorithm to coordinate these operations and construct efficient pipeline execution.

4.1 Sub-Stage Graph Construction

Proper abstraction granularity is fundamental for efficient end-to-end scheduling. Existing RL pipeline modeling methods [28, 45] operate at the stage level, where rollout, training, and reference forward are abstracted as atomic stages. This granularity hides intra-stage behavior. A single rollout stage contains high-utilization prefill bursts, low-utilization decoding and idles for external tool usage. Stage-level scheduling cannot distinguish them. Resource imbalance thus becomes invisible, and scheduling opportunities are lost.

Sub-Stage Graph Abstraction. JigsawRL models the RL pipeline as a sub-stage graph. As shown in Figure 5, the graph starts from a stage-level structure (rollout, training, reference inference, tool calling), covering a range of agentic

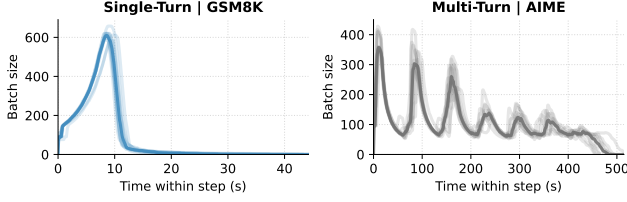


Figure 6: Decoding batch size variation across 10 adjacent steps in different pipelines using Qwen3-4B-Instruct, showing strong temporal consistency across steps.

RL pipelines [13, 16, 23, 49]. JigsawRL expands it into the *sub-stage graph* to expose both intra-stage and inter-worker imbalance.

- **Intra-stage:** We decompose the rollout stage based on the number of tokens processed per forward computation, including prefill tokens and active decoding requests. Token count determines computational density, so sub-stages defined this way have uniform resource profiles.
- **Inter-worker:** Different DP workers process heterogeneous samples. The sub-stage graph contains one replica per DP instance. Each replica tracks the sample subset assigned to its instance.

For each sub-stage, we need to profile for execution duration, memory footprint, batch size, and sequence length. These measurements guide multiplexing decisions. However, as discussed in §3.1, different pipelines exhibit dynamic execution behavior, which makes offline construction difficult.

Inter-Step Consistency. Despite these dynamics, we observe strong temporal similarity in rollout behavior over adjacent steps. Figure 6 shows the decoding batch size across 10 consecutive steps. In single-turn GSM8K, the degradation curves almost fully overlap. In multi-turn AIME, batch-size surges caused by repeated prefill bursts recur at similar positions. This temporal consistency arises because the training data distribution and the model’s capability evolves slowly across adjacent steps. JigsawRL therefore uses recent-step profiles to approximately construct and updates the sub-stage graph.

Profile-based Graph Construction. To instantiate the sub-stage graph, JigsawRL extracts sub-stage boundaries from recent execution profiles. For each forward step, JigsawRL computes its token count and maps it into workload buckets that correspond to distinct execution regimes with different compute and memory behavior. In practice, we use three buckets: $[0, 128)$, $[128, 1024)$, and $[1024, \infty)$. JigsawRL updates the current sub-stage only when the bucket assignment remains stable over a short window. Let B_k denote the bucket index of the k -th forward step. A transition is triggered only when the bucket remains unchanged for L_s consecutive steps. In practice, we set $L_s = 10$, which is sufficient to suppress

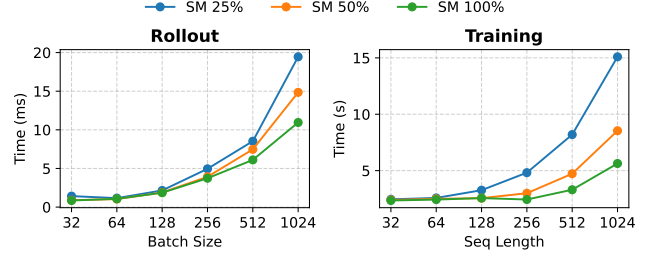


Figure 7: Latency comparison of Rollout and Training sub-stages when using different compute resources (SMs), evaluated on Qwen3-4B on 4 H100 GPUs.

fluctuations while preserving persistent shifts between sub-stages.

4.2 Sub-stage Multiplexing

JigsawRL multiplexes sub-stages from different graphs to improve resource utilization. The concurrent execution of two sub-stages introduces two key constraints. First, the sub-stages compete for GPU compute resources, leading to potential slowdown due to SM contention. Second, their combined memory footprint must fit within the available GPU memory. JigsawRL must adjust resource allocation at the sub-stage level to enable efficient multiplexing. Static and coarse-grained multiplexing frameworks [11, 30, 31] fall short in this setting.

Compute Resource Utilization. Different sub-stages exhibit distinct sensitivity to GPU compute resources (SMs). We study this by controlling SM numbers used by the kernel via NVIDIA MPS [30]. As shown in Figure 7, compute-intensive sub-stages, including Training and large-batch Rollout (≥ 256), are highly sensitive to SM allocation, with slowdown up to $2.7\times$ with 25% SMs. In contrast, small-batch Rollout (< 128) is largely insensitive to SM availability, indicating a memory-bound regime. These differences lead to asymmetric interference. As shown in Figure 8, multiplexing compute-bound sub-stages results in strong contention (slowdown up to $2.19\times$), while pairing compute-bound and memory-bound sub-stages yields better efficiency due to complementary resource usage. Based on this, JigsawRL prioritizes multiplexing complementary sub-stages and applies SM partitioning to mitigate compute resource contention.

GPU Memory Utilization. Memory footprint directly constrains both execution efficiency and multiplexing feasibility. Training is more sensitive to memory availability. As shown in Figure 9, during training, micro-batch size determines GPU memory usage and introduces a trade-off between throughput and memory consumption. During rollout, memory usage is largely driven by model weights and the KV cache, which bound the number of concurrent decoding requests. Under

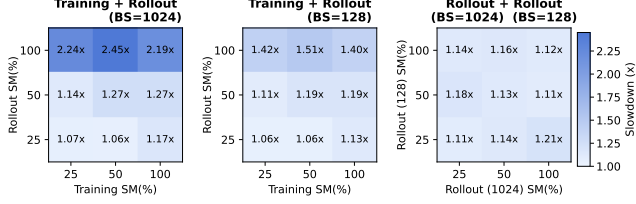


Figure 8: Slowdown under different SM partitioning when multiplexing two sub-stages (A + B). Each cell shows the slowdown of sub-stage A compared to isolated execution, evaluated on Qwen3-4B on 4 H100 GPUs.

large-model settings (e.g., serving Qwen3-32B on 4 H100 GPUs), we observe up to $1.43\times$ slowdown when reducing the memory consumption from 80% to 40%. Therefore, the GPU memory allocation for each sub-stage also becomes a key control knob for multiplexing.

Slowdown Modeling. Given the compute and memory constraints above, JigsawRL needs to predict how much slowdown each multiplexing choice incurs. However, exhaustively profiling multiplexing slowdown over all execution parameters is impractical. JigsawRL therefore constructs the slowdown model over a compressed space of sub-stage types and discrete resource budgets. We discretize the resource allocation space to keep profiling tractable. Specifically, compute allocation is discretized into four SM budgets (25%, 50%, 75%, and 100%), and memory allocation into four levels (20%, 40%, 60%, and 80% of the GPU memory budget). Combined with ~ 5 sub-stage types, this results in on the order of 10^2 measurements per pipeline, which is lightweight as an initialization cost. We denote the slowdown lookup function of sub-stage k_A when co-located with k_B as

$$s(k_A, k_B) = f(k_A, k_B, \alpha_A, Mem_A),$$

where α_A and Mem_A denote the compute and memory share assigned to k_A , with the remaining resources assigned to k_B . In practice, configurations with similar (α_A, Mem_A) exhibit similar slowdown behavior, allowing interpolation between neighboring profiled points. JigsawRL profiles f offline on this sparse grid and interpolates to estimate unseen configurations.

Dynamic Resource Allocation. To reduce compute resource contention, we leverage NVIDIA Green Context [32] to bound the SM resources available to individual sub-stages. Each process partitions the device SMs and creates streams that limit the maximum number of SMs according to the discrete budgets. For the rollout sub-stages, we pre-record multiple groups of CUDA graphs on these resource-bounded streams during inference server initialization [67]; at each decoding step, the system replays the CUDA graph corresponding to the SM budget determined by the current co-location scenario. For the reference and training sub-stages [65], we

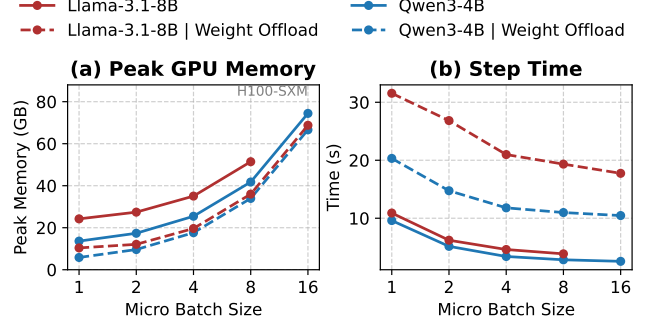


Figure 9: Impact of micro-batch size on per-GPU memory usage and training time on 4 H100 GPUs. Results are shown for both standard training and CPU weight offloading [38], with a fixed global batch size of 256.

directly bind the resource-bounded stream to the forward and backward operations. This coarse-grained discretization keeps the CUDA graph memory footprint manageable and avoids the wave quantization effects that arise when high-performance kernels (e.g., GEMMs, FlashAttention) are mapped to irregular SM counts [11, 27].

For GPU memory resources, JigsawRL dynamically adjusts the memory footprint to co-locate different sub-stages. For training and reference sub-stages, we tune the micro-batch size to trade off slowdown with memory usage. For rollout sub-stages, where memory allocation is fixed at launch time, JigsawRL periodically reconfigures the KV cache pool based on recent execution behavior (every ~ 30 steps in practice). The reconfiguration cost is amortized over multiple steps.

4.3 Sub-stage Merging

Rollout across DP workers introduces additional workload imbalance that is not resolved by intra-DP sub-stage multiplexing. Long-tail samples may leave a subset of DP workers executing low-utilization rollout while others have already finished. As shown in Figure 10(a), even at low utilization, these sub-stages still consume compute and memory resources, slowing down other concurrent sub-stages.

Long-Tail Sample Migration and Aggregation. JigsawRL handles such multiplexing inefficiency by dynamically migrating rollout samples across DP workers. As shown in Figure 10(b), JigsawRL continuously tracks the execution progress of rollout sub-stages during model training. When multiple DP workers are detected to be executing low-utilization rollout sub-stages, JigsawRL selects a target DP worker and migrates the long-tail rollout samples from other workers to it. This migration aggregates fragmented long-tail workloads onto a small number of DP workers. By leveraging the dynamic batching [63] capability of the rollout engine, the

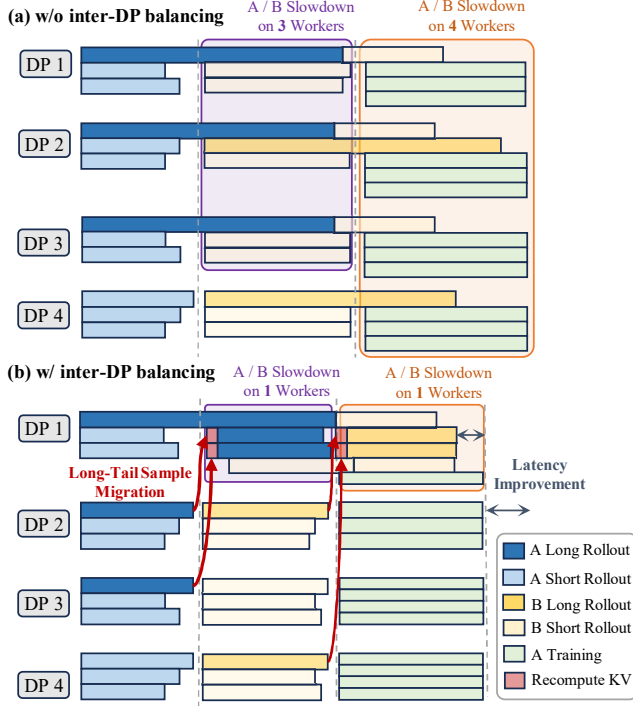


Figure 10: GigsawRL mitigates inter-worker imbalance by sample migration and multiplexing-aware workload balancing.

aggregated workloads achieve higher GPU utilization with very small latency increase. Meanwhile, the rest of DP workers can release resources for the current pipeline, enabling more effective sub-stage execution for other pipelines.

GigsawRL determines whether to perform migration based on the expected total execution time. Consider a set of low-utilization rollout sub-stages $\mathcal{K} = \{k_1, k_2, \dots, k_n\}$ on n DP workers. Suppose DP instance t is selected as the migration target. Each sub-stage k_i is currently multiplexed with another sub-stage k'_i . We estimate the expected execution time improvement of migrating all rollout samples in $\mathcal{K}$ to the target workers t as $\Delta T = T_{\text{origin}} - T_{\text{migr}}$, where

$$T_{\text{origin}} = \max_{i \in [1, n]} (s(k_i, k'_i)T(k_i)) + \max_{i \in [1, n]} (s(k'_i, k_i)T(k'_i)),$$

$$T_{\text{migr}} = T(\mathcal{K}) + \sum_{i=1}^n M(k_i) + \max(s(k'_t, \mathcal{K})T(k'_t), \max_{i \in [1, n], i \neq t} T(k'_i)).$$

Here, $T(\mathcal{K})$ denotes the execution time of the aggregated rollout workload. $s(k'_t, \mathcal{K})$ denotes the slowdown of k'_t under multiplexing with the aggregated workload. $M(k_i)$ denotes the migration overhead of sub-stage k_i . In practice, the migration cost is dominated by KV cache recomputation. We estimate

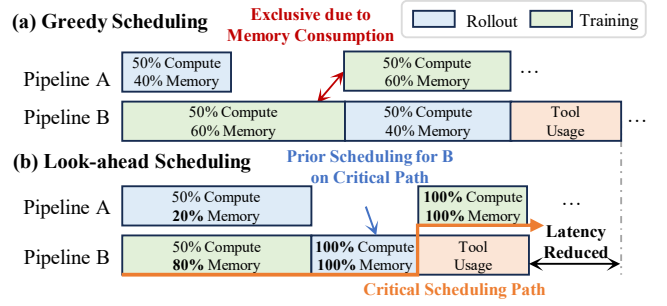


Figure 11: Comparison between greedy and critical-path-aware look-ahead scheduling.

this overhead based on the FLOPs of the migrated samples and the average MFU of the prefill stages.

Multiplexing-aware Workload Balancing. After migrating and aggregating long-tail rollout samples of pipeline A onto a specific DP instance t , GigsawRL further achieves workload balance across DP workers under multiplexing. When another pipeline B is under multiplexing, the uniform data parallelism or workload balancing in exclusive execution [52, 65] becomes suboptimal. This is because DP worker t may suffer from multiplexing slowdown, while other DP workers do not.

To address this, we introduce a multiplexing-aware load balancing strategy that dynamically skews the workload distribution. As illustrated in Figure 10, both the rollout sub-stages of B and training sub-stages of A assign less workload to DP worker t to compensate for its multiplexing interference. Let bs_i denote the batch size assigned to DP_i , and $T(bs)$ the exclusive execution time for a given batch size. To equalize execution time across all workers, we set:

$$\frac{T(bs_i)}{T(bs_t)} \approx s(k_{B,t}, k_A), \quad \forall i \neq t,$$

where $s(k_{B,t}, k_A)$ is the slowdown factor estimated by the model in §4.2. In practice, when Pipeline B is in the rollout sub-stage, we achieve this by routing fewer requests to the inference server on t . For reference or training sub-stages, we dynamically adjust the micro-batch sizes across DP workers.

4.4 Look-Ahead Scheduling

With the sub-stage graph and slowdown model, GigsawRL schedules sub-stages across co-located pipelines to minimize the total completion time. A greedy scheduler that minimizes only the immediate latency of the current co-location sub-stages can be globally suboptimal. Figure 11 illustrates such an example. Greedy scheduling allocates more resources to Pipeline A's rollout when multiplexing with Pipeline B's training. However, this prevents A's subsequent training from co-locating with B due to the memory constraints. In contrast, assigning fewer resources to A preserves this co-location opportunity.

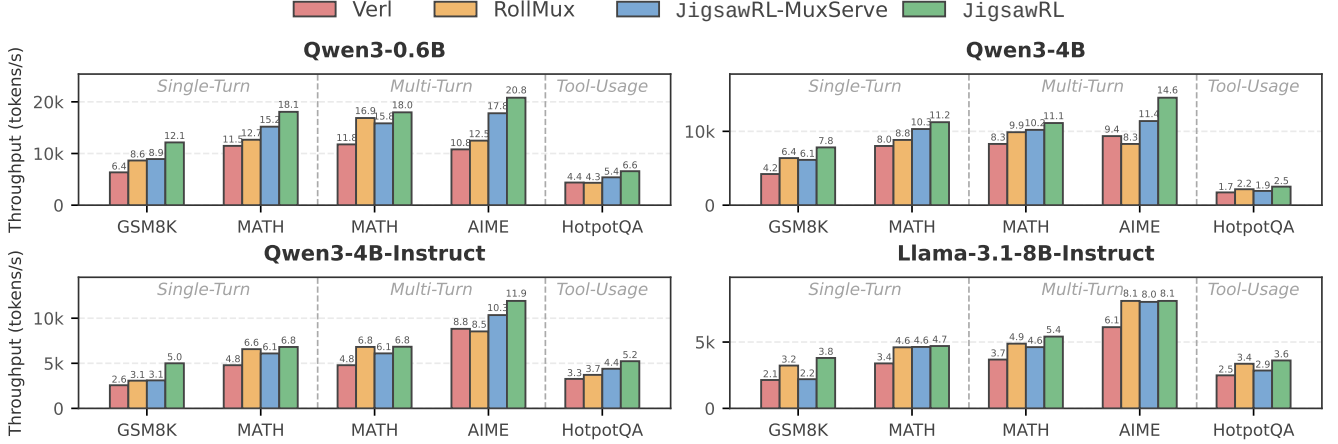


Figure 12: Throughput of homogeneous pipelines across different agentic RL pipelines on 8 H100 GPUs.

tunity. Moreover, since B’s rollout contains a tool-usage sub-stage, prioritizing its rollout allows the subsequent CPU-based tool calling to overlap with A’s training sub-stage, thereby further reducing the overall slowdown.

JigsawRL adopts a windowed look-ahead policy when choosing among candidate scheduling actions. At each scheduling point, JigsawRL maintains a frontier $\mathcal{F}$: the set of sub-stages that are ready to run across all co-located pipelines. On $\mathcal{F}$, JigsawRL considers two operations in §4.2 and §4.3: (i) multiplexing a pair $(k_A, k_B) \in \mathcal{F}$ on the same worker under a resource split r , and (ii) merging two sub-stages on different workers into one. Each candidate action a is thus paired with an allocation r , and its per-sub-stage execution time is derived from the slowdown model $s(k_A, k_B)$. Applying (a, r) retires the chosen sub-stages and admits their successors along the intra-pipeline dependency edges; JigsawRL then enumerates (a, r) over the updated frontier and repeats, much like fitting puzzle pieces together one at a time.

The cost of (a, r) is the critical path of $\mathcal{F}$, the longest chain of sub-stages that must execute sequentially:

$$C(a, r) = \max_{p \in \mathcal{P}(\mathcal{F})} \sum_{k \in p} \hat{T}_{a,r}(k),$$

where $\mathcal{P}(\mathcal{F})$ enumerates all dependency paths. This chain bounds how fast the window can finish, so minimizing $C(a, r)$ jointly accounts for the action and its allocation. JigsawRL picks the (a, r) with the smallest cost and continues scheduling. We use $W = 3$ by default, which we find offers a good balance between scheduling quality and search overhead.

5 Evaluation

5.1 Experimental Setup

Hardware Platforms. We conduct experiments on both single-node and multi-node settings. For single-node evaluation, we use a cluster with 8 H100 GPUs, each with 80 GB HBM3 memory, interconnected via NVLink. For multi-node evaluation, we evaluate scalability on an A100 cluster with 64 GPUs, where each node contains 4 GPUs with 80 GB HBM memory. GPUs within each node are connected via NVLink. **RL Pipelines.** We evaluate three representative RL pipelines for agent training. *Single-Turn* generates a complete response in one step, commonly used for reasoning tasks [16, 21]. *Multi-Turn* iteratively produces intermediate outputs for self-refinement and multi-agent interaction [8, 25, 41, 53]. *Tool-Usage* invokes external tools during rollout; we use the Search-R1 pipeline [23]. We adopt GRPO [43] for policy optimization, with a global batch size of 64, group size 4, and maximum response length of 8192 tokens.

Datasets. For single-turn pipelines, we use math and reasoning datasets, including GSM8K [9] and MATH [18]. For multi-turn pipelines, we additionally include the challenging AIME dataset. For tool-usage pipelines, we use the multi-hop QA dataset HotpotQA [58] for RAG-based agent training. We use the MS MARCO [6] dataset as the external RAG database and build an IVF4096 ($nprobe=32$) index with FAISS [10].

Models. We use base models including Qwen3-0.6B and Qwen3-4B, and instruct-tuned models including Qwen3-4B-Instruct and Llama-3.1-8B-Instruct. For larger-scale evaluation, we use DeepSeek-R1-Distill-14B, a Qwen2.5-14B model distilled from DeepSeek-R1 [16], and the base model Qwen3-32B. These models span small- to medium-scale sizes with varying levels of fine-tuning and reasoning capabilities; in practice, instruct-tuned and distilled models exhibit

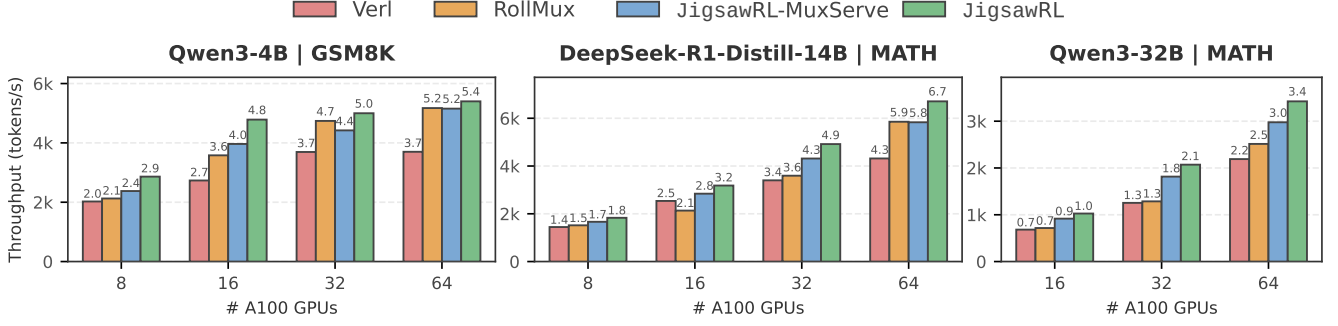


Figure 13: Throughput comparison between Verl and JigsawRL scaling from 8 to 64 A100 GPUs with larger models.

stronger capabilities on the tasks in our test datasets.

Baselines. For synchronous RL, we use Verl [45] and RollMux [55] as baselines. Verl adopts a rollout-training with co-located workers on a same set of GPUs, while RollMux uses time-multiplexing with disaggregated workers. We also include a version of JigsawRL using MuxServe [11] to multiplex the rollout stages, noted as JigsawRL-MuxServe. For asynchronous RL, we use StreamRL [68] and AReal [13] as the baseline. StreamRL overlaps rollout and training stages on disaggregated workers with one-step bounded staleness. AReal further adopts fully asynchronous execution, tolerating staleness up to several model update steps.

For backend implementations, we use SGLang [67] for rollout and FSDP [65] for training, which are well-suited for agentic workloads and small- to mid-scale models. JigsawRL also supports vLLM [24] and Megatron [46]. Existing frameworks such as Slime [71] and NeMo [1] follow similar designs by integrating state-of-the-art rollout and training engines.

5.2 Synchronous Pipeline Settings

We first evaluate the main deployment setting, where two synchronous RL pipelines with the same model architecture and datasets but different weights, are executed concurrently on shared GPUs. We define throughput as the aggregated number of tokens processed by all the pipelines per unit time.

Overall Performance Improvement. We measure the throughput improvement of JigsawRL over Verl across different agentic pipelines. As shown in Figure 12, JigsawRL consistently achieves the highest throughput, with an average speedup of $1.56\times$ over Verl and up to $1.95\times$. Compared with RollMux and PuzzRL-MuxServe, JigsawRL further improves throughput by $1.27\times$ and $1.23\times$ on average, respectively.

First, JigsawRL achieves larger speedups over Verl with more severe rollout imbalance. For example, for the weaker model Qwen3-0.6B, the speedup on GSM8K is higher than on MATH, since the more difficult dataset tends to induce consistently long reasoning and generation, thus reducing

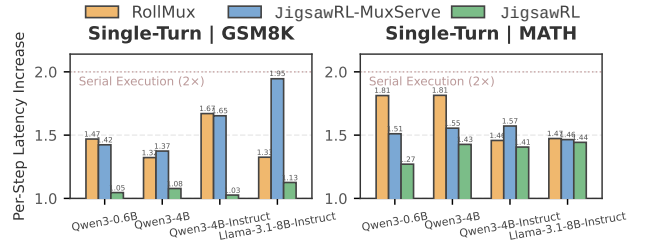


Figure 14: Latency increase of different multiplexing methods compared to the exclusive execution of Verl on 8 H100 GPUs.

imbalance and limiting the optimization opportunity.

Second, the speedups over RollMux are more strongly correlated with workload imbalance across stages. For example, in the multiturn AIME workload with Qwen3-4B, the rollout stage accounts for 71.1%, leading to imbalance across workers. In contrast, for Llama-3.1-8B-Instruct, rollout and training are more balanced, with 56.3% and 43.7% of the total execution time, respectively. In such cases, RollMux and JigsawRL achieve comparable throughput.

Scalability. To evaluate JigsawRL at scale, we conduct experiments on 8–64 A100 GPUs using larger models. As illustrated in Figure 13, JigsawRL consistently scales across different model sizes and cluster sizes, achieving up to $1.75\times$ throughput improvement over Verl and RollMux with an average of $1.47\times$ and $1.31\times$, respectively. This scalability comes from two factors. First, as the cluster size increases, existing systems suffer from lower utilization and more pronounced stage imbalance, creating more opportunities for spatial multiplexing across complementary stages. Second, JigsawRL further mitigates the long-tail effect at higher DP degrees through inter-DP-worker sample migration, which helps maintain balanced execution and sustain throughput gains at scale.

Latency Trade-off. While JigsawRL improves aggregate throughput through multiplexing, it also increases the per-

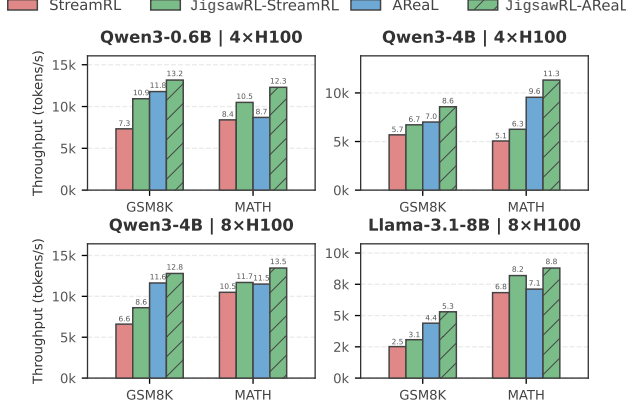


Figure 15: Throughput comparison of JigsawRL under one-step off-policy (StreamRL) and fully asynchronous RL (AReaL).

step latency of each individual pipeline due to multiplexed execution. Figure 14 shows that this overhead remains moderate: the latency increase is $1.48\times$ on average and can be as low as $1.14\times$, substantially smaller than the $2\times$ latency cost of running two pipelines serially. We further observe that the latency increase is smaller on easier task sets such as GSM8K, which exhibit higher intra-batch imbalance.

5.3 Diverse Pipeline Settings

We further evaluate JigsawRL on additional pipeline settings, including asynchronous pipelines and heterogeneous pipelines.

Asynchronous RL. Figure 15 evaluates JigsawRL under one-step off-policy [68] and fully asynchronous RL [13], using disaggregated rollout and training workers with equal resource allocation. JigsawRL achieves an average speedup of $1.25\times$ (up to $1.54\times$) over StreamRL and $1.21\times$ (up to $1.41\times$) over AReaL. These results show that JigsawRL remains effective in asynchronous settings, mitigating inter-worker imbalance and low utilization during decoding. Concretely, each worker enables multiplexing of the training stage of one pipeline with the rollout stage of another, thereby improving utilization.

Heterogeneous Models. We also evaluate JigsawRL with pipelines of different model sizes. As illustrated in Figure 16, JigsawRL successfully improves the aggregated throughput with the individual pipeline slowdown limited to at most $1.34\times$. Interestingly, we observe a higher improvement when co-locating models have size disparities (Qwen3-0.6B and Llama-3.1-8B). This is because the sub-stages of the smaller model can better fit into the long-tail rollout sub-stages of the larger LLM and improve the utilization.

Synchronous and Asynchronous Pipelines. We further evaluate JigsawRL by multiplexing synchronous pipeline with

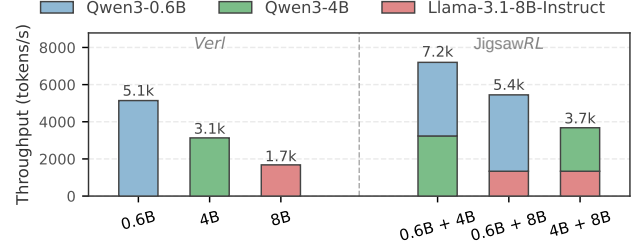


Figure 16: Throughput of heterogeneous pipelines in Verl and JigsawRL on 4 H100 GPUs. The training dataset is GSM8K.

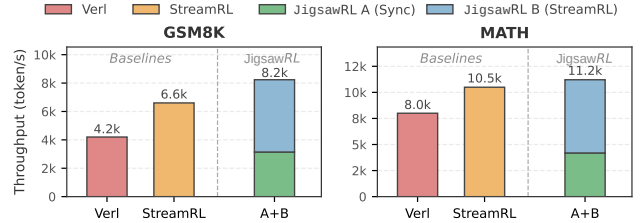


Figure 17: Throughput of JigsawRL multiplexing synchronous (A) and one-step off-policy asynchronous (B) pipeline on 8 H100 GPUs. The model used is Qwen3-4B.

an asynchronous one (StreamRL [68]). As illustrated in Figure 17, JigsawRL improves the aggregated throughput by $1.24\times$ over Verl and StreamRL on GSM8K, and $1.07\times$ on MATH. The improvement is higher on GSM8K because both pipelines exhibit more severe rollout imbalance under easier tasks, leaving more under-utilized periods that JigsawRL can fill through sub-stage multiplexing. In contrast, MATH produces longer and more uniform rollouts in both pipelines, reducing the multiplexing opportunity.

5.4 Case and Ablation Studies

In this section, we study the compatibility of JigsawRL with existing parallelism methods, as well as the effectiveness of individual optimization methods.

Integration with Other Parallelism Methods. We evaluate JigsawRL with Data Parallelism (DP) and Tensor Parallelism (TP). Figure 18 illustrates the throughput-cost trade-off space.

In Verl, scaling with TP in a single node (TP=2 to TP=4) yields marginal gains, while cross-node scaling (TP=8) is limited by communication overhead due to the bandwidth gap between intra-node NVLink and inter-node networks. In contrast, DP is the more efficient parallelism for multi-node scaling: increasing from (TP=4, DP=1) to (TP=4, DP=4) achieves a $1.2\times$ throughput improvement, but incurs a $3.4\times$ cost increase due to exacerbated imbalance.

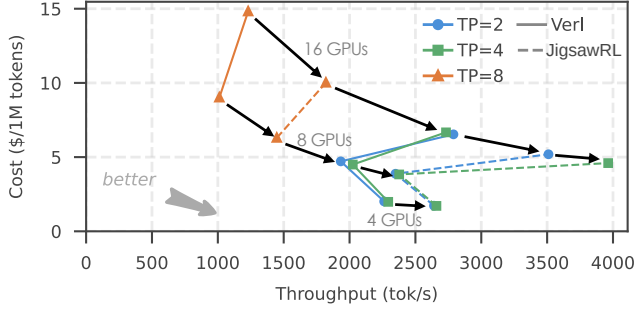


Figure 18: Throughput-Cost trade-off for Qwen3-4B scaling from 4 to 16 GPUs. The training dataset is GSM8K.

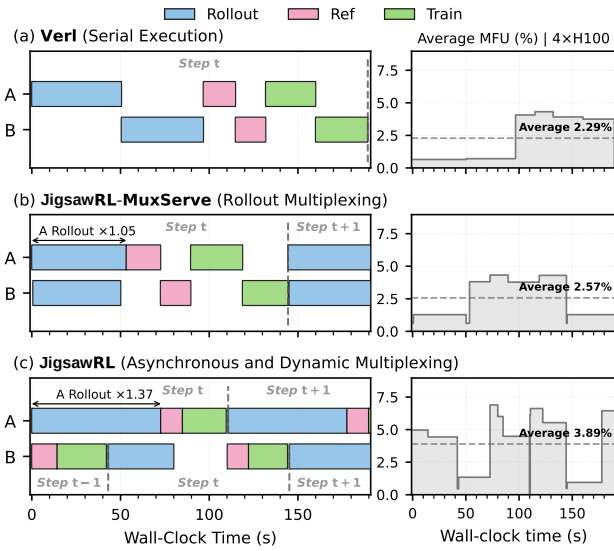


Figure 19: Pipeline multiplexing strategies and their impact on average MFU (Single-Turn, Qwen3-4B, 4xH100, GSM8K).

In contrast, our proposed Pipeline Multiplexing introduces a new, cost-efficient parallelism dimension. Under identical TP and DP settings, JigsawRL improves the aggregated throughput by $1.16\times$ to $1.48\times$ without cost efficiency penalties. The flatter slope of its scaling curve demonstrates potential for larger-scale deployments. Furthermore, JigsawRL is orthogonal to automated parallelism search frameworks like ReaL [28], enabling further optimizations atop the ideal parallelism.

Dynamic Sub-Stage Multiplexing. We evaluate the effectiveness of our sub-stage-level multiplexing by comparing different pipeline execution strategies, as shown in Figure 19. Verl’s serial execution (a) exhibits prolonged low MFU during rollout due to long-tail effects. Using MuxServe for rollout multiplexing (b) partially improves overlap, but still suffers from

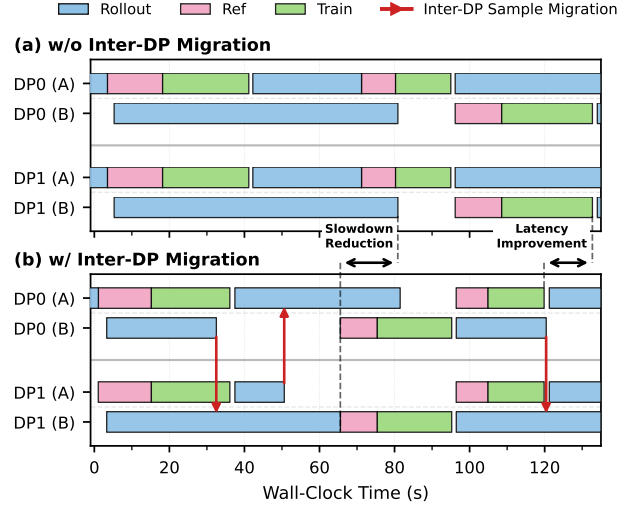


Figure 20: Inter-DP workload migration and its impact on execution latency (Single-Turn, Qwen3-4B, 4xH100, GSM8K)

low rollout utilization and increased latency due to serialized training. In contrast, JigsawRL (c) enables asynchronous, fine-grained sub-stage multiplexing. By dynamically co-locating memory-bound rollout with compute-bound training and reference, it improves GPU utilization by $1.7\times$ and $1.5\times$ over Verl and JigsawRL-MuxServe strategies, respectively.

Inter-worker Workload Migration and Balancing. We evaluate the effectiveness of our long-tail sample migration and multiplexing-aware workload balancing mechanisms. As depicted in Figure 20(a), without inter-DP migration, the long-tail samples of Pipeline A’s rollout stage span across all DP instances. This execution continuously interferes with Rollout B. In contrast, in Figure 20(b), by migrating the long-tail rollout samples of Pipeline A from DP1 to DP0, JigsawRL effectively isolates the long-tail rollout sub-stage to different DP instances, alleviate the interference of sub-stage with extremely low utilization. Such optimization translates into the observable reduction in the overall training step latency improvement for both Pipeline A and B.

6 Conclusion

We present JigsawRL, a cost-efficient RL post-training framework built on *Pipeline Multiplexing*, a new dimension of RL parallelism. JigsawRL models each pipeline as a sub-stage graph that exposes imbalance hidden by stage-level systems, and formulates pipeline multiplexing as a graph scheduling problem solved with a look-ahead heuristic. On up to 64 GPUs across three agentic RL pipelines and six models, JigsawRL achieves up to $1.85\times$ throughput over synchronous baselines and $1.54\times$ over asynchronous baselines, while sup-

porting heterogeneous pipelines with moderate latency overhead.

Our current study focuses on small- to mid-scale dense models, which remain the mainstream choice for agentic RL among individual researchers and smaller teams. Extending JigsawRL to MoE architectures and LoRA-style parameter-efficient training opens further opportunities: MoE exposes expert-level imbalance that enables finer-grained multiplexing, while LoRA adapters over a shared base model naturally form multiplexing candidates with complementary memory footprints. We leave these directions to future work.

References

- [1] Nemo rl: A scalable and efficient post-training library. <https://github.com/NVIDIA-NeMo/RL>, 2025. GitHub repository.
- [2] Alibaba qwen model downloads: Metrics and enterprise impact. <https://www.aicerts.ai/news/alibaba-qwen-model-downloads-metrics-and-enterprise-impact/>, 2026.
- [3] Marwa Abdulhai, Isadora White, Charlie Snell, Charles Sun, Joey Hong, Yuexiang Zhai, Kelvin Xu, and Sergey Levine. Lmrl gym: Benchmarks for multi-turn reinforcement learning with language models. *arXiv preprint arXiv:2311.18232*, 2023.
- [4] Amazon Web Services. Amazon EC2 On-Demand Pricing. <https://aws.amazon.com/ec2/pricing/on-demand/>. Accessed: 2026-03-21.
- [5] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.
- [6] Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, et al. Ms marco: A human generated machine reading comprehension dataset. *arXiv preprint arXiv:1611.09268*, 2016.
- [7] Qiaoling Chen, Zijun Liu, Peng Sun, Shenggui Li, Guoteng Wang, Ziming Liu, Yonggang Wen, Siyuan Feng, and Tianwei Zhang. Respec: Towards optimizing speculative decoding in reinforcement learning systems. *arXiv preprint arXiv:2510.26475*, 2025.
- [8] Yixing Chen, Yiding Wang, Siqi Zhu, Haofei Yu, Tao Feng, Muhan Zhang, Mostofa Patwary, and Jiaxuan You. Multi-agent evolve: Llm self-improve through co-evolution. *arXiv preprint arXiv:2510.23595*, 2025.
- [9] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [10] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. *IEEE Transactions on Big Data*, 2025.
- [11] Jiangfei Duan, Runyu Lu, Haojie Duanmu, Xiuhong Li, Xingcheng Zhang, Dahua Lin, Ion Stoica, and Hao Zhang. Muxserve: flexible spatial-temporal multiplexing for multiple llm serving. *arXiv preprint arXiv:2404.02015*, 2024.
- [12] Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjuan Zhong. Retool: Reinforcement learning for strategic tool use in llms. *arXiv preprint arXiv:2504.11536*, 2025.
- [13] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, et al. Areal: A large-scale asynchronous reinforcement learning system for language reasoning. *arXiv preprint arXiv:2505.24298*, 2025.
- [14] Wei Gao, Yuheng Zhao, Dakai An, Tianyuan Wu, Lunxi Cao, Shaopan Xiong, Ju Huang, Weixun Wang, Siran Yang, Wenbo Su, et al. Rollpacker: Mitigating long-tail rollouts for fast, synchronous rl post-training. *arXiv preprint arXiv:2509.21009*, 2025.
- [15] Wei Gao, Yuheng Zhao, Tianyuan Wu, Shaopan Xiong, Weixun Wang, Dakai An, Lunxi Cao, Dilxat Muhtar, Zichen Liu, Haizhou Zhao, et al. Rollart: Scaling agentic rl training via disaggregated infrastructure. *arXiv preprint arXiv:2512.22560*, 2025.
- [16] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shitong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [17] Jingkai He, Tianjian Li, Erhu Feng, Dong Du, Qian Liu, Tao Liu, Yubin Xia, and Haibo Chen. History doesn't repeat itself but rollouts rhyme: Accelerating reinforcement learning with rhymerrl. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 929–945, 2026.
- [18] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob

- Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- [19] Jian Hu, Xibin Wu, Zilin Zhu, Weixun Wang, Dehao Zhang, Yu Cao, et al. Openrlhf: An easy-to-use, scalable and high-performance rlhf framework. *arXiv preprint arXiv:2405.11143*, 6, 2024.
- [20] Zhengding Hu, Vibha Murthy, Zaifeng Pan, Wanlu Li, Xiaoyi Fang, Yufei Ding, and Yuke Wang. Hedrag: Co-optimizing generation and retrieval for heterogeneous rag workflows. In *Proceedings of the ACM SIGOPS 31st symposium on operating systems principles*, pages 623–638, 2025.
- [21] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.
- [22] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.
- [23] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Serkan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-rl: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.
- [24] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023.
- [25] Zhuofeng Li, Haoxiang Zhang, Seungju Han, Sheng Liu, Jianwen Xie, Yu Zhang, Yejin Choi, James Zou, and Pan Lu. In-the-flow agentic system optimization for effective planning and tool use. In *International Conference on Learning Representations (ICLR)*, 2026.
- [26] Bingshuai Liu, Ante Wang, Zijun Min, Liang Yao, Haibo Zhang, Yang Liu, Anxiang Zeng, and Jinsong Su. Spec-rl: Accelerating on-policy reinforcement learning via speculative rollouts. *arXiv preprint arXiv:2509.23232*, 2025.
- [27] Jiamin Lu, Jingwei Sun, Yunlong Xu, Peng Sun, and Guangzhong Sun. Conco: Optimizing compilation of concurrent tensor programs on shared gpu. In *Proceedings of the 39th ACM International Conference on Supercomputing*, pages 640–653, 2025.
- [28] Zhiyu Mei, Wei Fu, Kaiwei Li, Guangju Wang, Huanchen Zhang, and Yi Wu. Real: Efficient rlhf training of large language models with parameter reallocation. *Proceedings of Machine Learning and Systems*, 7, 2025.
- [29] Michael Noukhovitch, Shengyi Huang, Sophie Xhonneux, Arian Hosseini, Rishabh Agarwal, and Aaron Courville. Asynchronous rlhf: Faster and more efficient off-policy rl for language models. *arXiv preprint arXiv:2410.18252*, 2024.
- [30] NVIDIA. Multi-process service. <https://docs.nvidia.com/deploy/mps/index.html>, 2022.
- [31] NVIDIA. Nvidia mig (multi-instance gpu). <https://www.nvidia.com/en-us/technologies/multi-instance-gpu/>, 2022.
- [32] NVIDIA Corporation. *CUDA Driver API: Green Contexts*, 2025. Accessed: 2025-03-29.
- [33] OpenPipe. Serverless rl. <https://openpipe.ai/blog/serverless-rl>, 2024. Accessed: 2026-03-04.
- [34] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- [35] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37:126544–126565, 2024.
- [36] Ruoyu Qin, Weiran He, Weixiao Huang, Yangkun Zhang, Yikai Zhao, Bo Pang, Xinran Xu, Yingdi Shan, Yongwei Wu, and Mingxing Zhang. Seer: Online context learning for fast synchronous llm reinforcement learning. *arXiv preprint arXiv:2511.14617*, 2025.
- [37] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023.
- [38] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: international conference for high performance computing, networking, storage and analysis*, pages 1–16. IEEE, 2020.
- [39] Daniel Ritter, Owen Oertell, Bradley Guo, Jonathan Chang, Kianté Brantley, and Wen Sun. Llm can learn to reason via off-policy rl. *arXiv preprint arXiv:2602.19362*, 2026.

- [40] Ning Shang, Yifei Liu, Yi Zhu, Li Lyna Zhang, Weijiang Xu, Xinyu Guan, Buze Zhang, Bingcheng Dong, Xudong Zhou, Bowen Zhang, et al. rstar2-agent: Agentic reasoning technical report. *arXiv preprint arXiv:2508.20722*, 2025.
- [41] Lior Shani, Aviv Rosenberg, Asaf Cassel, Oran Lang, Daniele Calandriello, Avital Zipori, Hila Noga, Orgad Keller, Bilal Piot, Idan Szpektor, et al. Multi-turn reinforcement learning with preference human feedback. *Advances in Neural Information Processing Systems*, 37:118953–118993, 2024.
- [42] Zelei Shao, Vikranth Srivatsa, Sanjana Srivastava, Qingyang Wu, Alpary Ariyak, Xiaoxia Wu, Ameen Patel, Jue Wang, Percy Liang, Tri Dao, Ce Zhang, Yiyang Zhang, Ben Athiwaratkun, Chenfeng Xu, and Junxiong Wang. Beat the long tail: Distribution-aware speculative decoding for rl training. *arXiv preprint arXiv:2511.13841*, 2025.
- [43] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [44] Guangming Sheng, Yuxuan Tong, Borui Wan, Wang Zhang, Chaobo Jia, Xibin Wu, Yuqi Wu, Xiang Li, Chi Zhang, Yanghua Peng, et al. Laminar: A scalable asynchronous rl post-training framework. *arXiv preprint arXiv:2510.12633*, 2025.
- [45] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297, 2025.
- [46] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [47] Xin Tan, Yicheng Feng, Yu Zhou, Yimin Jiang, Yibo Zhu, and Hong Xu. Orchestrl: Dynamic compute and network orchestration for disaggregated rl. *arXiv preprint arXiv:2601.01209*, 2026.
- [48] Thinking Machines. Tinker documentation: Serverless api framework for modular rl. <https://tinker-docs.thinkingmachines.ai/>, 2024. Accessed: 2026-03-04.
- [49] Kangrui Wang*, Pingyue Zhang*, Zihan Wang*, Yaning Gao*, Linjie Li*, Qineng Wang, Hanyang Chen, Chi Wan, Yiping Lu, Zhengyuan Yang, Lijuan Wang, Ranjay Krishna, Jiajun Wu, Li Fei-Fei, Yejin Choi, and Manling Li. Vagen:reinforcing world model reasoning for multi-turn vlm agents, 2025.
- [50] Siqi Wang, Hailong Yang, Junjie Zhu, Xuezhu Wang, Yufan Xu, and Depei Qian. Rlhfspec: Breaking the efficiency bottleneck in rlhf training via adaptive drafting. *arXiv preprint arXiv:2512.04752*, 2025.
- [51] Weixun Wang, Shaopan Xiong, Gengru Chen, Wei Gao, Sheng Guo, Yancheng He, Ju Huang, Jiaheng Liu, Zhendong Li, Xiaoyang Li, et al. Reinforcement learning optimization for large-scale learning: An efficient and user-friendly scaling library. *arXiv preprint arXiv:2506.06122*, 2025.
- [52] Zheng Wang, Anna Cai, Xinfeng Xie, Zaifeng Pan, Yue Guan, Weiwei Chu, Jie Wang, Shikai Li, Jianyu Huang, Chris Cai, et al. {WLB-LLM}:{Workload-Balanced} 4d parallelism for large language model training. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 785–801, 2025.
- [53] Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, et al. Ragen: Understanding self-evolution in llm agents via multi-turn reinforcement learning. *arXiv preprint arXiv:2504.20073*, 2025.
- [54] Rui Wei, Hanfei Yu, Shubham Jain, Yogarajan Sivakumar, Devsh Tiwari, Jian Li, Seung-Jong Park, and Hao Wang. Rlhflex: Serverless computing for efficient rlhf. *arXiv preprint arXiv:2602.22718*, 2026.
- [55] Tianyuan Wu, Lunxi Cao, Yining Wei, Wei Gao, Yuheng Zhao, Dakai An, Shaopan Xiong, Zhiqiang Lv, Ju Huang, Siran Yang, et al. Rollmux: Phase-level multiplexing for disaggregated rl post-training. *arXiv preprint arXiv:2512.11306*, 2025.
- [56] Yongji Wu, Xueshen Liu, Haizhong Zheng, Juncheng Gu, Beidi Chen, Z Morley Mao, Arvind Krishnamurthy, and Ion Stoica. RLboost: Harvesting preemptible resources for cost-efficient reinforcement learning on llms. *arXiv preprint arXiv:2510.19225*, 2025.
- [57] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

- [58] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pages 2369–2380, 2018.
- [59] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757, 2022.
- [60] Zhewei Yao, Reza Yazdani Aminabadi, Olatunji Ruwase, Samyam Rajbhandari, Xiaoxia Wu, Ammar Ahmad Awan, Jeff Rasley, Minjia Zhang, Conglong Li, Connor Holmes, et al. Deepspeed-chat: Easy, fast and affordable rlhf training of chatgpt-like models at all scales. *arXiv preprint arXiv:2308.01320*, 2023.
- [61] Chenhao Ye, Huaizheng Zhang, Mingcong Han, Baoquan Zhong, Xiang Li, Qixiang Chen, Xinyi Zhang, Weidong Zhang, Kaihua Jiang, Wang Zhang, et al. Tensorhub: Scalable and elastic weight transfer for llm rl training. *arXiv preprint arXiv:2604.09107*, 2026.
- [62] Chao Yu, Yuanqing Wang, Zhen Guo, Hao Lin, Si Xu, Hongzhi Zang, Quanlu Zhang, Yongji Wu, Chunyang Zhu, Junhao Hu, et al. Rlinf: Flexible and efficient large-scale reinforcement learning via macro-to-micro flow transformation. *arXiv preprint arXiv:2509.15965*, 2025.
- [63] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX symposium on operating systems design and implementation (OSDI 22)*, pages 521–538, 2022.
- [64] Shan Yu, Jiarong Xing, Yifan Qiao, Mingyuan Ma, Yangmin Li, Yang Wang, Shuo Yang, Zhiqiang Xie, Shiyi Cao, Ke Bao, et al. Prism: Unleashing gpu sharing for cost-efficient multi-llm serving. *arXiv preprint arXiv:2505.04021*, 2025.
- [65] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. Pytorch fsdp: experiences on scaling fully sharded data parallel. *arXiv preprint arXiv:2304.11277*, 2023.
- [66] Haizhong Zheng, Jiawei Zhao, and Beidi Chen. Prosperity before collapse: How far can off-policy rl reach with stale data on llms? *arXiv preprint arXiv:2510.01161*, 2025.
- [67] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.
- [68] Yinmin Zhong, Zili Zhang, Xiaoni Song, Hanpeng Hu, Chao Jin, Bingyang Wu, Nuo Chen, Yukun Chen, Yu Zhou, Changyi Wan, et al. Streamrl: Scalable, heterogeneous, and elastic rl for llms with disaggregated stream generation. *arXiv preprint arXiv:2504.15930*, 2025.
- [69] Yinmin Zhong, Zili Zhang, Bingyang Wu, Shengyu Liu, Yukun Chen, Changyi Wan, Hanpeng Hu, Lei Xia, Ranchen Ming, Yibo Zhu, et al. Optimizing {RLHF} training for large language models with stage fusion. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 489–503, 2025.
- [70] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.
- [71] Zilin Zhu, Chengxing Xie, Xin Lv, and slime Contributors. slime: An llm post-training framework for rl scaling. <https://github.com/THUDM/slime>, 2025. GitHub repository. Corresponding author: Xin Lv.